to configure it. But you cannot do both. If you don't include the `action` attribute or any custom routing attributes, the form will submit to the current page's URL. For the vast majority of CRUD situations, that is most likely what you will do, and you will process the submission in the `OnPost` handler of the same page that generated the form. You will often use the tag helper attributes to set the form's action when the processing page is a different page or the form is processed within a named handler. You already saw, for example, where you might want to set the page handler name when we covered named handlers in chapter 3:

```
<form method="post" asp-page-handler="Search">
```

The form tag helper supports one additional custom attribute: `antiforgery`. This takes a Boolean value to control whether the request verification token is rendered or not, so it provides a way for you to disable request verification on a form-by-form basis selectively. If the attribute is omitted, the value defaults to `true`. Thinking back to the example where we covered named handlers in chapter 3, the search form in listing 3.32 could have been written to use a form action tag helper instead of setting the submission URL via the form tag helper. The following listing demonstrates how to use the `page-handler` attribute to generate a `formaction` attribute in the rendered button.

Listing 6.1 The form action tag helper

```
<form method="post">
    <p>Search</p>
    <input name="searchTerm" />
    <button asp-page-handler="Search">Search</button>    ❶
</form>
```

❶ The page-handler attribute is applied to the button, resulting in a formaction attribute being rendered to the browser.